

① Write a program in C to find the second smallest element in an array.

Code → #include <stdio.h>
#include <limits.h>

```
int main() {  
    int n, i;  
    printf("Enter no. of elements: ");  
    scanf("%d", &n);  
    int arr[n];  
    printf("Enter elements: \n");  
    for (i = 0; i < n; i++)  
        scanf("%d", &arr[i]);  
  
    int smallest = INT_MAX, secondsmallest = INT_MAX;  
    for (i = 0; i < n; i++) {  
        if (arr[i] < smallest) {  
            secondsmallest = smallest;  
            smallest = arr[i];  
        }  
        else if (arr[i] < secondsmallest &&  
                arr[i] != smallest) {  
            secondsmallest = arr[i];  
        }  
    }  
  
    if (secondsmallest == INT_MAX)  
        printf("No second smallest element \n");  
}
```

else

printf("second smallest element = %d \n",
secondsmallest);

return;

}

Output

Enter number of elements : 5

Enter elements:

21

65

59

21

67

Second smallest element = 27

② write a C program to find the sum of left diagonals of a matrix

→ #include <stdio.h>

```
int main() {
```

```
    int n, i, j, sum = 0;
```

```
    printf("Enter order of square matrix:");
```

```
    scanf("%d", &n);
```

```
    int matrix[n][n];
```

```
    printf("Enter matrix elements: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &matrix[i][j]);
```

```
    }
```

```
    for (i = 0; i < n; i++) {
```

```
        sum += matrix[i][i];
```

```
    printf("Sum of left Diagonal = %d\n", sum);
```

```
    return 0;
```

```
}
```

Output

Enter order of square matrix: 2

Enter matrix elements:

2 3

4 6

Sum of left diagonal = 8

find sum of rows and columns of a matrix

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int r, c, i, j;
```

```
    printf("Enter number of rows and columns: ");
```

```
    scanf("%d %d", &r, &c);
```

```
    int matrix[r][c];
```

```
    printf("Enter matrix elements :\n");
```

```
    for (i=0; i<r; i++){
```

```
        for (j=0; j<c; j++){
```

```
            scanf("%d", &matrix[i][j]);
```

```
        }
```

```
    for (i=0; i<r; i++){
```

```
        int rowsum=0;
```

```
        for (j=0; j<c; j++){
```

```
            rowsum += matrix[i][j];
```

```
            printf("sum of row %d = %d\n", i+1, rowsum);
```

```
        }
```

```
    for (j=0; j<c; j++){
```

```
        int colsum=0;
```

```
        for (i=0; i<r; i++){
```

```
            colsum += matrix[i][j];
```

```
            printf("sum of column %d = %d\n", j+1, colsum);
```

```
        }
```

```
    return 0;
```

```
}
```

output-

Enter no. of rows and columns: 2 2

Enter matrix elements:

4

6

7

9

Sum of Row 1 = 10

Sum of Row 2 = 16

Sum of Column 1 = 11

Sum of Column 2 = 15

④ Count the duplicate element in an array

→ #include <stdio.h>

```
int main()
```

```
{ int n, i, j, count = 0;
```

```
printf("Enter no. of elements ");
```

```
scanf("%d", &n);
```

```
int arr[n], visited[n];
```

```
printf("Enter elements ");
```

```
for (i = 0; i < n; i++)
```

```
scanf("%d", &arr[i]);
```

```
visited[i] = 0;
```

```
{
```

```
for (j = 0; j < n; j++)
```

```
{ if (visited[j] == 1) {
```

```
continue;
```

```
int duplicate = 0;
```

```
for (i = 0; i < n; i++)
```

```
{ if (arr[i] == arr[j])
```

```
duplicate++;
```

```
visited[i] = 1;
```

```
}
```

```
}
```

```
if (duplicate)
```

```
count++;
```

```
}
```

```
printf("%d duplicate element", count);
```

return

}

output →

Enter number of elements :

Enter elements :

23

54

67

12

54

68

Total duplicate elements = 1

⑤ Write a program in C to find the second largest element in an array.

```
→ #include <stdio.h>
#include <limits.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter no. of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter elements: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int largest = INT_MIN, secondLargest = INT_MIN;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (arr[i] > largest) {
```

```
            secondLargest = largest;
```

```
            largest = arr[i];
```

```
        }
```

```
        else if (arr[i] > secondLargest &&
```

```
            arr[i] != largest) {
```

```
            secondLargest = arr[i];
```

```
        if (secondLargest == INT_MIN)
```

```
            printf("No second largest element: \n");
```

```
        else
```

```
            printf("second largest element = %d \n", secondLargest);
```

```
    }
    return 0;
```

Output →

Enter number of elements : 5

Enter elements:

5

45

87

2

67

Second largest Element = 67

write a C program to delete an element at a desired position in an array

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, pos;
```

```
    printf("Enter no. of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter elements: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    printf("Enter position to delete (1 to %d): ", n);
```

```
    scanf("%d", &pos);
```

```
    if (pos < 1 || pos > n) {
```

```
        printf("Invalid position! \n");
```

```
    } else {
```

```
        for (i = pos - 1; i < n - 1; i++) {
```

```
            arr[i] = arr[i + 1];
```

```
        printf("Array after deletion: \n");
```

```
        for (i = 0; i < n - 1; i++)
```

```
            printf("%d", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

output →

Enter no. of elements : 5

Enter elements :

54

64

35

73

46

Enter position to delete (1 to 5) : 2

Array after deletion :

54 35 73 46

~~Q10~~

Lab Program 1

Write a C program to simulate the following CPU scheduling algorithm
a) FCFS b) SJF

FCFS (non preemptive)

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    int at[20], bt[20], ct[20], tat[20], wt[20],  
        rt[20];
```

```
    float avg-wt = 0, avg-tat = 0, avg-rt = 0;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival time & Burst time: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P%d AT: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d BT: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    for (i = 1; i < n; i++) {
```

```
        if (ct[i-1] < at[i])
```

```
            ct[i] = at[i] + bt[i];
```

```
        else
```

```

        ct[i] = ct[i-1] + bt[i];
    }

    for (i=0; i<n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = wt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt[i];
    }

    printf("\n process \t AT \t BT \t CT \t TAT \t WT \t RT \t\n");
    for (i=0; i<n; i++) {
        printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\n Average Waiting Time: %.2f", avg_wt/n);
    printf("\n Average Turn around time: %.2f", avg_tat/n);
    printf("\n Average Response Time: %.2f", avg_rt/n);
    return 0;
}

```

output →

Enter no. of processes:

Enter Arrival time and Burst time:

P1 AT: 0

P1 BT: 5

P2 AT: 1

P2 BT: 3

P3 AT: 2

P3 BT: 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	4	12	10	6	6

Average waiting time = 3.33

Average Turnaround Time = 7.33

Average Response Time = 3.33



b) SJF (non-preemptive)

→ #include <stdio.h>

#include <limits.h>

int main() {

int n, i, time = 0, completed = 0;

int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];

int done[20] = {0};

float avg-wt = 0, avg-tat = 0, avg-rt = 0;

printf("Enter no. of processes:");

scanf("%d", &n);

printf("Enter Arrival time & Burst time: \n");

for (i = 0; i < n; i++) {

printf("P%d AT: ", i+1);

scanf("%d", &at[i]);

printf("P%d BT: ", i+1);

scanf("%d", &bt[i]);

}

while (completed < n) {

int shortest = -1;

int min_bt = INT_MAX;

for (i = 0; i < n; i++) {

if (at[i] <= time & done[i] == 0 & bt[i] < min_bt)

min_bt = bt[i];

shortest = i;

}

}

if (shortest == -1) {

P2 AT: 1

P2 BT: 3

P3 AT: 2

P3 BT: 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	4	12	10	6	6

Average waiting time = 3.33

Average Turnaround time = 7.33

Average Response time = 3.33

8/2

b) SJF (non-preemptive)

→ #include <stdio.h>

#include <limits.h>

int main() {

int n, i, time = 0, completed = 0;

int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];

int done[20] = {0};

float avg-wt = 0, avg-tat = 0, avg-rt = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

printf("Enter Arrival time & Burst time: \n");

for (i = 0; i < n; i++) {

printf("P%d AT: ", i+1);

scanf("%d", &at[i]);

printf("P%d BT: ", i+1);

scanf("%d", &bt[i]);

}

while (completed < n) {

int shortest = -1;

int min_bt = INT_MAX;

for (i = 0; i < n; i++) {

if (at[i] <= time && done[i] == 0 && bt[i] < min_bt)

min_bt = bt[i];

shortest = i;

}

}

if (shortest == -1) {

time++;

continue

}

rt[shortest] = time - at[shortest];

time = bt[shortest];

ct[shortest] = time;

tat[shortest] = ct[shortest] - at[shortest];

wt[shortest] = tat[shortest] - bt[shortest];

done[shortest] = 1;

completed;

avg_wt += wt[shortest];

avg_tat += tat[shortest];

avg_rt += rt[shortest];

}

printf("In Process AT BT CT TAT WT RT \n");

for (i=0; i<n; i++)

printf("P%d. %d %d %d %d %d %d \n",

i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

}

printf("In Avg Waiting Time = %.2f", avg_wt/n);

printf("In Avg Turnaround Time = %.2f", avg_tat/n);

printf("In Avg Response time = %.2f \n", avg_rt/n);

return 0;

}

output →

Enter no. of process: 3

Enter Arrival Time and Burst Time:

P1 AT: 0

P1 BT: 5

P2 AT: 1

P2 BT: 3

P3 AT: 2

P3 BT: 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	4	12	10	6	6

Average Waiting Time = 3.33

Avg turnaround time = 7.33

Avg Response Time = 3.33



SJF (pre-emptive)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main()
```

```
{  
    int n, i, time = 0, completed = 0;
```

```
    int at[20], bt[20], rem-bt[20];
```

```
    int ct[20], fct[20], wt[20], rt[20];
```

```
    int first[20] = {0};
```

```
    float avg-wt = 0, avg-tat = 0, avg-rt = 0;
```

```
    printf("Enter no. of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival Time & Burst Time : \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P %d AT :", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P %d BT:", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        rem-bt[i] = bt[i];
```

```
        rt[i] = -1;
```

```
    }
```

```
    while (completed < n) {
```

```
        int shortest = -1;
```

```
        int min-bt = INT_MAX;
```

```
        for (i = 0; i < n; i++) {
```

```

rem-bt[i] < min-bt){
    min-bt = rem-bt[i];
    shortest = i;
}
}
if (shortest == -1)?
    timu++;
    continue
}
if (rt[shortest] == -1)
    rt[shortest] = timu - at[shortest];
rem-bt[shortest]--;
timu++;
if (rem-bt[shortest] == 0){
    completed++;
    ct[shortest] = timu;
    tat[shortest] = bt[shortest] - ct[shortest];
    wt[shortest] = tat[shortest] * bt[shortest];
    avg-wt += wt[shortest];
    avg-tat += tat[shortest];
    avg-rt += rt[shortest];
}
}
}

```

```

printf("InProcess AP BP CTAT WT RT \n");
for (i=0; i<n; i++){
    printf("P %d %d %d %d %d %d %d \n",
        i+1, at[i], bt[i], tat[i], wt[i], rt[i],

```

```

printf("In Average waiting time = %.2f", avg-wt);
printf("In Average turnaround time = %.2f", avg-tat);
printf("In Average response time = %.2f", avg-rt);
return 0;
}

```

Output →

Enter no. of processes: 3

Enter Arrival time and Burst time:

P1 AT: 0

P1 BT: 5

P2 AT: 1

P2 BT: 3

P3 AT: 2

P3 BT: 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	8	8	3	0
P2	1	3	4	3	0	0
P3	2	4	12	10	6	6

Average waiting time: 3.00

Average Turnaround time: 7.00

Average Response time: 2.00

- Between FCFS and SJF, the more efficient is SJF.

Reason → SJF runs the shortest job first, so small processes finish quickly and do not wait behind for long processes.

8/2
13/3

Lab Program 2.

Write a C program to simulate following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & non-preemptive)

Pre-emptive

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    int at[20], bt[20], pr[20], rem_bt[20];
```

```
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival Time, Burst Time & Priority:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P%d AT: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d BT: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
printf("P%d Priority:", i+1);
```

```
scanf("%d", &pr[i]);
```

```
rem_bt[i] = bt[i];
```

```
rt[i] = -1;
```

```
}
```

```
while(completed < n)
```

```
{
```

```
    int highest = -1;
```

```
    int min_pr = INT_MAX;
```

```
    for(i=0; i<n; i++) {
```

```
        if (rt[i] <= 0 && rem_bt[i] > 0 &&
```

```
            pr[i] < min_pr) {
```

```
                min_pr = pr[i];
```

```
                highest = i;
```

```
            }
```

```
        }
```

```
        if (highest == -1) {
```

```
            time ++;
```

```
            continue;
```

```
        }
```

```
        if (rt[highest] == 0) {
```

```
            completed ++;
```

```
            rt[highest] = time;
```

```
            tat[highest] = rt[highest] + at[highest];
```

```
wt[highest] = tat[highest] - bt[highest];
```

```
avg-wt += wt[highest];
```

```
avg-tat += tat[highest];
```

```
avg-rt += rt[highest];
```

```
}
```

```
}
```

```
printf("\nProcess AT BT PR CT TAT WT RT\n");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
printf("P%d AT%d BT%d PR%d CT%d TAT%d WT%d RT%d",
```

```
i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i],
```

```
rt[i],
```

```
})
```

```
printf("\nAvg Waiting time = %.2f", avg-wt/n);
```

```
printf("\nAvg Turnaround Time = %.2f", avg-tat/n);
```

```
printf("\nAvg Response time = %.2f\n", avg-rt/n);
```

```
return 0;
```

```
}
```

Output →

Enter no. of Processes: 3

Enter Arrival Time, Burst Time and Priority:

P1 AT: 0

P1 BT: 5

P1 Priority: 2

P2 AT: 2

P2 BT: 4

P2 Priority: 1

P3 AT: 3

P3 BT: 4

P3 Priority: 3

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	5	2	9	9	4	0
P2	2	4	1	4	4	0	0
P3	3	4	3	10	10	6	6

Avg waiting time = 3.33

Avg Turnaround Time = 7.67

Avg Response time = 2.00

Priority (non-preemptive)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    int at[20], bt[20], pr[20];
```

```
    int ct[20], tct[20], wt[20], rt[20];
```

```
    int done[20] = {0};
```

```
    float avg-wt = 0, avg-tct = 0, avg-rt = 0;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival time, Burst time and Priority:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P%d AT: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d BT: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        printf("P%d Priority: ", i+1);
```

```
        scanf("%d", &pr[i]);
```

```
    }
```

Date _____
Page _____

```
while (completed < n) {
```

```
    int highest = -1;
```

```
    int min-pr = INT_MAX;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (at[i] <= time && done[i] == 0 &&  
            pr[i] < min-pr) {
```

```
            min-pr = pr[i];
```

```
            highest = i;
```

```
        }
```

```
    }
```

```
    if (highest == -1) {
```

```
        time++;
```

```
        continue;
```

```
    }
```

```
    rt[highest] = time - at[highest];
```

```
    time += bt[highest];
```

```
    ct[highest] = time;
```

```
    tat[highest] = ct[highest] - at[highest];
```

```
    wt[highest] = tat[highest] - bt[highest];
```

```
    avg-wt += wt[highest];
```

```
    avg-tat += tat[highest];
```

```
    avg-rt += rt[highest];
```

```
done[highest] = 1;  
completed++;
```

```
}
```

```
printf("\nProcess AT BT PR CT TAT WT RT\n");
```

```
for(i=0; i<n; i++){
```

```
printf("P%d %d %d %d %d %d %d\n",  
i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i],  
rt[i]);
```

```
}
```

```
printf("\n Average Waiting Time = %.2f", avg-wt/n);
```

```
printf("\n Average Turnaround Time = %.2f", avg-tat/n);
```

```
printf("\n Average Response Time = %.2f", avg-rt/n);
```

```
return 0;
```

```
}
```

Output →

Enter no. of processes: 3

Enter Arrival Time, Burst Time & Priority:

P1 AT: 0

P1 BT: 5

P1 Priority: 3

P2 AT: 1

P2 BT: 3

P₂ Priority: 1

P₃ AT: 2

P₃ BT: 3

P₃ Priority: 2

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	5	3	5	5	0	0
P2	1	3	1	8	7	4	4
P3	2	3	2	11	9	6	6

Average Waiting Time = 3.33

Average Turnaround Time = 7.00

Average Response Time = 3.33

~~8/3~~
2.13

Lab 3 Programs

Q. Round Robin (with different quantum size)

Code →

```
#include <stdio.h>
```

```
void roundRobin (int n, int at[], int bt[], int tq) {  
    int rt[n], ct[n], wt[n], tot[n];
```

```
    for (int i=0; i<n; i++) {  
        rt[i] = bt[i];  
    }
```

```
    int time = 0, completed = 0;
```

```
    while (completed < n) {  
        int idle = 1;
```

```
        for (int i=0; i<n; i++) {  
            if (at[i] <= time & rt[i] > 0) {  
                idle = 0;  
                if (rt[i] > tq) {  
                    time += tq;  
                    rt[i] -= tq;  
                } else {
```

```
                    time += rt[i];
```

```

        ct[i] = time;
        rt[i] = 0;
        completed++;
    }
}

if (idle) {
    time++;
}
}

float total_wt = 0, total_tat = 0;

for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += resp[i];
}

printf("In Time Quantum = %d\n", tq);
printf("Process\tAT\tBT\tCT\tWT\tTAT\n");

for (int i = 0; i < n; i++) {
    printf("P %d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], wt[i], tat[i]);
}

printf("Average WT = %.2f\n", total_wt/n);
printf("Average TAT = %.2f\n", total_tat/n);

```

```
printf("Average RT = %.2f \n", total_rt/n);
```

```
{  
int main() {
```

```
int n;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n];
```

```
for (int i = 0; i < n; i++) {
```

```
printf("Process P%d \n", i+1);
```

```
printf("Arrival time:");
```

```
scanf("%d", &at[i]);
```

```
printf("Burst time:");
```

```
scanf("%d", &bt[i]);
```

```
}
```

```
int num_q, tq;
```

```
printf("Enter no. of different time quantum:");
```

```
scanf("%d", &num_q);
```

```
for (int i = 0; i < num_q; i++) {
```

```
printf("Enter time quantum: %d:", i+1);
```

```
scanf("%d", &tq);
```

```
roundRobin(n, at, bt, tq);
```

```
}
```

```
return 0;
```

```
}
```

Output →

Enter number of processes: 3

Process P1

Arrival Time: 1

Burst Time: 3

Process P2

Arrival Time: 1

Burst Time: 4

Process P3

Arrival Time: 1

Burst Time: 5

Enter no. of different time quantum: 2

Enter time quantum 1: 3

Time quantum: 3

Process	AT	BT	CT	WT	TAT	RT
P1	1	3	4	0	3	0
P2	1	4	11	6	10	3
P3	1	5	13	7	12	6

Average WT = 4.33

Average TAT = 8.33

Average RT = 3.00

Enter Time Quantum 2:5

Time Quantum = 5

Process	AT	BT	CT	WT	TAT	RT
P1	1	3	4	0	3	10
P2	1	4	8	3	7	3
P3	1	5	13	7	12	7

Average WT = 3.33

Average TAT = 7.33

Average RT = 9.33

Now there two TQs 3 & 5.

TQ = 3

WT and TAT are higher since there is more switching (multiple rounds) due to lesser TQ.

TQ = 5

Now WT & TAT are lower since all processes gets completed in single round (larger TQ) and hence it is more efficient.

Thus increasing TQ reduces switching and WT, TAT decreases thus making it more efficient.

2/13

WAP to simulate multi-level queue scheduling algo. considering the following scenario. All the processes in the system are divided into two categories - system processes & user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code →

```
typedef struct {  
    int process-id;  
    int arrival-time;  
    int burst-time;  
    int remaining-time;  
    int type;  
    int completion-time;  
    int waiting-time;  
    int turnaround-time;  
    int started;  
    int start-time;  
} Process;
```

```
typedef struct {  
    int items[MAX-PROCESSES];  
    int front, rear;  
} Queue;
```

```
void initQueue (Queue* q) {
```

```
    q->front = 0;
```

```
    q->rear = -1;
```

```
}
```

```
int isEmpty (Queue* q) {
```

```
    return q->rear < q->front;
```

```
}
```

```
void enqueue (Queue* q, int value) {
```

```
    if (q->rear < MAX_PROCESSES-1) {
```

```
        q->rear++;
```

```
        q->items[q->rear] = value;
```

```
}
```

```
}
```

```
int dequeue (Queue* q) {
```

```
    if (!isEmpty(q)) {
```

```
        int val = q->items[q->front];
```

```
        q->front++;
```

```
        return val;
```

```
}
```

```
return -1;
```

```
}
```

```
int compareArrival (const void* a, const void* b) {
```

```
    Process* p1 = (Process*) a;
```

```
    Process* p2 = (Process*) b;
```

```

        if (p1->arrival-time < p2->arrival-time)
            return -1;
        else if (p1->arrival-time > p2->arrival-time)
            return 1;
        else
            return 0;
    }

```

```

int main() {

```

```

    int n;

```

```

    Process processes[MAX-PROCESSES];

```

```

    printf("Enter no. of processes:");

```

```

    scanf("%d", &n);

```

```

    for (int i=0; i<n; i++) {

```

```

        printf("Process %d\n", i);

```

```

        printf("Enter Arrival time:");

```

```

        scanf("%d", &processes[i].arrival-time);

```

```

        printf("Enter Burst time:");

```

```

        scanf("%d", &processes[i].burst-time);

```

```

        printf("Enter type 0=system, 1=user:");

```

```

        scanf("%d", &processes[i].type);

```

```

        processes[i].process-id=i;

```

```

        processes[i].remaining-time=processes[i].burst-time;

```

```

        processes[i].start=0;

```

```

        processes[i].start-time=-1; }

```

```
qsort (processes, n, sizeof (Process, compareArrival),
```

```
queue systemQueue, userQueue;
```

```
initQueue (&systemQueue);
```

```
initQueue (&userQueue);
```

```
while (completed < n) {
```

```
    while (i < n && process[i].current-time) {
```

```
        if (processes[i].type == 0)
```

```
            enqueue (&system, i);
```

```
        else
```

```
            enqueue (&userQueue, i);
```

```
        i++
```

```
    }
```

```
    if (current-process == NULL) {
```

```
        if (!isEmpty (&systemQueue))
```

```
            int id1 = dequeue (&systemQueue)
```

```
        else if (!isEmpty (&userQueue))
```

```
            int id2 = dequeue (&userQueue)
```

```
        else {
```

```
            current-time++;
```

```
            continue;
```

```
    }
```

```
if (current-process->remaining-time == 0) {
```

```
    current-process->completion-time = current-time;
```

```
}
```

double total-wt=0, total-tat=0;

```
for (int j=0; j<n; j++)
```

```
printf ("%d\t%d\t%.1t\t%.1d\t%.1d\n",
```

```
process[j].process-id;
```

```
process[j].type == 0);
```

```
process[j].arrival-time;
```

```
process[j].burst-time;
```

```
process[j].com-time;
```

```
total-wt += process[j].waiting-time;
```

```
total-tat += process[j].turnaround-time;
```

```
}
```

```
printf ("Avg WT: %.2f\n", total-wt/n);
```

```
printf ("Avg TAT: %.2f\n", total-tat/n);
```

```
}
```

Output ->

Enter no. of processes = 4

Process: 0

Enter Arrival time: 0

Enter Burst time: 5

Enter type (0=system, 1=user): 1

Process 1

Enter Arrival time: 2

Enter Burst time: 3

Enter type (0=system, 1=user): 1

Process 2

Enter Arrival time: 1

Enter Burst time: 4

Enter type (0=system, 1=user): 0

Process 3

Enter Arrival Time: 3

Enter Burst Time: 2

Enter Type (0=system, 1=user): 1

ID	Type	AT	BT	CT	WT	TA
0	system	0	5	5	0	5
1	system	1	4	9	4	8
2	user	2	3	12	7	10
3	user	3	2	14	9	11

Avg. WT : 5.00

Avg TA : 8.50

~~27/13~~

Q-

Write a C program to simulate Real-time CPU scheduling algorithms-

- Rate Monotonic
- Earliest deadline first

Code →

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 10
```

```
typedef struct {
```

```
    int id;
```

```
    int burst;
```

```
    int deadline;
```

```
    int period;
```

```
    int weight;
```

```
    int ct, wt, tat;
```

```
} Process;
```

```
void edf(Process P[], int n) {
```

```
    float util = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        float util = 0;
```

```
        util += (float) p[i].burst / p[i].deadline;
```

```

for (int i=0; i<n-1; i++)
    for (int j=i+1; j<n; j++)
        if (p[i].deadline > p[j].deadline) {
            Process t = p[i];
            p[i] = p[j];
            p[j] = t;
        }

```

```

int time = 0;
for (int i=0; i<n; i++) {
    time += p[i].burst;
    p[i].ct = time;
    p[i].tat = p[i].ct;
    p[i].wt = p[i].tat - p[i].burst;
}

```

```

printf(" EDF Scheduling == \n");
printf(" Utilization = %.2f \n", util);
printf(" Schedulable if  $U \leq 1$  \n");

```

```

printf(" ID\tBT\tDL\tCT\tWT\tTAT \n");
for (int i=0; i<n; i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d \n", p[i].id,
        p[i].burst, p[i].deadline, p[i].ct, p[i].wt);

```

```

void rms (Process p[], int n) {
    float util = 0;

```

```
util += (float) p[i].burst / p[i].period;
```

```
float bound = n * (pow(2, (float) 1/n) - 1);
```

```
for (int i = 0; i < n; i++)
```

```
for (int j = i + 1; j < n; j++)
```

```
if (p[i].period > p[j].period) {
```

```
    Process t = p[i];
```

```
    p[i] = p[j];
```

```
}
```

```
int time = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    time += p[i].burst;
```

```
    p[i].ct = time;
```

```
    p[i].tat = p[i].ct;
```

```
    p[i].wt = p[i].tat - p[i].burst;
```

```
}
```

```
for printf("==== RMS EDF scheduling ====\n");
```

```
printf("Utilization = %.2f\n", util);
```

```
printf("RM Bound if use RM = %.4f\n", bound);
```

```
printf("ID BT D deadline CT WT TAT\n");
```

```
for (int i = 0; i < n; i++) { if (util < bound) {  
    printf("schedulable");  
    else printf("not guaranteed schedulable");
```

```
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        p[i].id, p[i].burst, p[i].deadline, p[i].ct,
```

```
        p[i].wt, p[i].tat);
```

```
void main() {  
    float total_weight = 0;
```

```
    for (int i = 0; i < n; i++)  
        total_weight += p[i].weight;
```

```
    printf("n====Proportional Scheduling====n");  
    printf("Total weight = %d\n", total_weight);  
    printf("ID\tweight\tCPU share\n");
```

```
    for (int i = 0; i < n; i++) {  
        float share = (float) p[i].weight / total_weight;  
        printf("%d\t%d\t%.2f\n", p[i].id, p[i].weight,  
                                                       share);  
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    Process p1[MAX], p2[MAX], p3[MAX];
```

```
    for
```

```
        printf("Enter no. of processes:");
```

```
        scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("In Process %d\n", i);
```

```
        p1[i].id = p2[i].id = p3[i].id = i;
```

```
scanf("%d", &p1[i].burst);
```

```
printf("Deadline (Dmax): ");
```

```
scanf("%d", &p1[i].deadline);
```

```
printf("Period (RMS): ");
```

```
scanf("%d", &p1[i].period);
```

```
printf("Weight (Proportional): ");
```

```
scanf("%d", &p1[i].weight);
```

```
p2[i] = p1[i];
```

```
p3[i] = p1[i];
```

```
}
```

```
edf(p1, n);
```

```
rms(p2, n);
```

```
proportional(p3, n);
```

```
return 0;
```

```
}
```

Output

Output →

Enter no. of processes: 3

Process 0

Burst time: 3

Deadline (EDF): 7

Period (RMS): 20

Weight (Proportional): 5

Process 1

Burst time: 2

Deadline (EDF): 4

Period (RMS): 5

Weight (Proportional): 7

Process 2

Burst Time: 2

Deadline (EDF): 8

Period (RMS): 10

Weight (Proportional): 4

=== EDF Scheduling ===

Utilization = 1.18

Schedulable if $U \leq 1$

ID	BT	DL	CT	WT	TAT
1	2	4	2	0	2
0	3	7	2	5	5

2 2 8 7 5 7

=== RMS Scheduling ===

utilization = 0.75

RM Bound = 0.7798

Schedulable

ID	BT	Period	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

=== Proportional Scheduling ===

Total weight = 16

ID	weight	CPU share
0	5	0.31
1	7	0.44
2	4	0.25

gkz
pol

Lab-5

- Write a C program to simulate -
- Producer-Consumer problem using semaphores
 - Dining-Philosophers problem

Code →

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 3
#define ITEMS 15

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Produced: %d at buffer [%d]\n", i, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
}
```

```
        usleep(100000);  
    }  
    return NULL;  
}
```

```
void *consumer(void *arg){  
    for(int i=0; i<ITEMS; i++){  
        sem_wait(&full);  
        pthread_mutex_unlock(&mutex);  
        sem_post(&empty);  
        usleep(100000);  
    }  
    return NULL;  
}
```

```
int main() {
```

```
    pthread_t p, c;
```

```
    sem_init(&empty, 0, BUFFER_SIZE);
```

```
    sem_init(&full, 0, 0);
```

```
    pthread_mutex_init(&mutex, NULL);
```

```
    pthread_create(&p, NULL, producer, NULL);
```

```
    pthread_create(&c, NULL, consumer, NULL);
```

```
    pthread_join(p, NULL);
```

```
    pthread_join(c, NULL);
```

```
    sem_destroy(&empty);
```

```
    sem_destroy(&full);
```

```
    pthread_mutex_destroy(&mutex);
```

```
    return 0;
```

Output →

Produced: 0 at buffer[0]

Consumed: 0 from buffer[0]

Produced: 1 at buffer[1]

Consumed: 1 from buffer[1]

Produced: 2 at buffer[2]

Consumed: 2 from buffer[2]

Produced: 3 at buffer[0]

Produced: 4 at buffer[1]

Consumed: 13 from buffer[1]

Consumed: 14 from buffer[2]

Trace →

Step	Operation	Buffer state	in	out	empty	full
1	Producer inserts 0	[0 - -]	1	0	2	1
2	Consumer removes 0	[- - -]	1	1	3	0
3	Producer inserts 1	[- 1 -]	2	1	2	1
4	Consumer removes 1	[- - -]	2	2	3	0
5	Producer inserts 2	[- - 2]	3	2	2	1
6	Consumer removes 2	[- - -]	3	3	3	0
7	Producer inserts 3	[3 - -]	1	0	2	1
8	Producer inserts 4	[3 4 -]	2	0	1	2
9	Consumer removes 3	[- 4 -]	2	1	2	1
10	Producer inserts 5	[- 4 5]	0	1	1	2
11	Consumer removes 4	[- - -]				

12	Producer inserts 6	[6-5]						
13	Producer inserts 7	[6 7 5]	1	2	1	2	1	
14	Consumer removes 5	[6 7 -]	2	2	0	3	1	
15	Consumer removes 6	[7-]	2	0	1	2	1	
16	Producer inserts 8	[7 8]	2	1	2	1	1	
17	Consumer removes 7	[-- 8]	0	1	1	2	1	
18	Producer inserts 9	[9- 8]	1	2	2	1	1	
19	Consumer removes 8	[9--]	1	0	1	2	1	
20	Consumer removes 9	[---]	1	1	2	1	1	
21	Producer inserts 10	[10-]	2	1	3	0	1	
22	Consumer removes 10	[---]	2	2	2	1	1	
23	Producer inserts 11	[11-]	0	2	3	0	1	
24	Consumer removes 11	[---]	0	0	2	1	1	
25	Producer inserts 12	[12--]	1	0	3	0	1	
26	Consumer removes 12	[---]	1	1	2	1	1	
27	Producer inserts 13	[13-]	2	1	3	0	1	
28	Consumer removes 13	[---]	2	2	2	1	1	
29	Producer inserts 14	[14-]	0	2	3	0	1	
30	Consumer removes 14	[---]	0	0	2	1	1	

Tracy

Code →

```
#include <stdio.h>
#include <pthread.h>
#include <
```

Tracy →

Step	Philosopher	Action	Fork State [0-4]
1	P ₀	Thinking	(F, F, F, F, F)
2	P ₀	Picks left fork 0	[0 F F F F]
3	P ₀	Picks right fork 1	[0 0 F F F]
4	P ₀	Eating	[0 0 F F F]
5	P ₀	Puts fork 0 & 1	[F F F F F]
6	P ₁	Thinking	[F F F F F]
7	P ₁	Picks left fork 1	[F 1 1 F F F]
8	P ₁	Picks right fork 2	[F 1 1 F F]
9	P ₁	Eating	[F 1 1 F F]
10	P ₁	Puts fork 1 & 2	[F F F F F]
11	P ₃	Thinking	[F F F F F]
12	P ₃	Pick left 3	[F F F 3 F]
13	P ₃	Pick right 4	[F F F 3 3]
14	P ₃	Get	[F F F 3 3]
15	P ₃	Put back 3 & 4	[F F F F F]
16	P ₂	Think	[F F F F F]

18	P_2	Pick right 3	$[F F 2 2 F]$
19	P_2	bat	$[F F 2 2 F]$
20	P_2	Put both 2 & 3	$[F F F F F]$
21	P_4	think	$[F F F F F]$
22	P_4	Pick left 0	$[4 F F F F]$
23	P_4	Pick right 4	$[4 F F F 4]$
24	P_4	bat	$[4 F F F 4]$
25	P_4	Put back 0 & 4	$[F F F F F]$

✓
~~8/2~~
 24/4

Q- Write a C program for deadlock avoidance & detect

→ #include <stdio.h>

int main() {

int n, m, i, j, k;

printf("Enter ^{total} number of processes: ");

scanf("%d", &n);

printf("Enter no. of resource types: ");

scanf("%d", &m);

int Allocation[n][m], Max[n][m], Need[n][m];

int Available[m], work[m], finish[n], safe[n];

printf("Enter Allocation Matrix:");

for(i=0; i<n; i++){

for(j=0; j<m; j++){

scanf("%d", &Allocation[i][j]);

}

printf("Enter Max Matrix:");

for(i=0; i<n; i++){

for(j=0; j<m; j++){

scanf("%d", &Max[i][j]);

}

printf("Enter Available Resources:");

for(i=0; i<m; i++){

scanf("%d", &Available[i]);

for(i=0; i<n; i++){

printf("1. Safety Algo in 2. Resource Request Allocation");
 printf("Enter choice (1 or 2):");
 scanf("%d", &ch);
 switch (ch) {

case 1: { for (i=0; i<m; i++)
 work[i]=available[i];

for (i=0; i<n; i++) {

finish[i]=0;

int count=0;

while (count < n)

int found=0;

for (i=0; i<n; i++) {

if (finish[i]==0) {

for (j=0; j<m; j++)

if (need[i][j] > work[j])

break;

if (j==m) {

for (k=0; k<m; k++)

work[k] += alloc[i][k];

Safe Count++ = k =

finish[i]=1;

found=1;

}

}

if (found==0)

break;

}

if (count == n) {

SAFE STATE in seq sequence: 1;

```
case 2: { int p, req[m];  
    printf("Enter process number machine: ");  
    scanf("%d", &p);  
    printf("Enter request vector: ");  
    for (i=0; i<m; i++)
```

```
        if (req[i] > need[p][i]) {
```

```
            printf("Error: Exceeded Need");
```

```
            return 0; }
```

```
    if (req[i] > available[i]) {
```

```
        printf("Resources not available");
```

```
        return;
```

```
    }
```

```
    for (i=0; i<m; i++) {
```

```
        available[i] -= req[i];
```

```
        allocation[i] += req[i];
```

```
        need[p][i] = -req[i];
```

```
    }
```

```
    printf("In Request Granted");
```

```
    break;
```

```
}
```

```
default: printf("Invalid choice");
```

```
}
```

```
return 0;
```

```
}
```

Output →

Enter no. of processes: 5

Enter no. resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Max Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resource:

3 3 2

1. Safety Algorithm

2. Resource Request

Enter Choice 1

SAFE STATE

Safe sequence: P1 P3 P4 P0 P2

Enter choice 2

Enter process number: 1

Enter request vector:

1 0 2

```
#include <stdio.h>
```

```
int n, m, i, j, k;
```

```
printf("Enter no. of process: ");
```

```
scanf("%d", &n);
```

```
printf("Enter no. of resource types: ");
```

```
scanf("%d", &m);
```

```
int Alloc[n][m], Request[n][m];
```

```
int avail[m];
```

```
printf("Enter Allocation Matrix:\n");
```

```
scanf("%d", &Alloc[i][j]);
```

```
}
```

```
printf("Enter Request Matrix:\n");
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<m; j++)
```

```
scanf("%d", &Request[i][j]);
```

```
}
```

```
}
```

```
printf("Enter Avail Matrix:");
```

```
for (i=0; i<n; i++)
```

```
scanf("%d", &Avail[i]);
```

```
int Work[m], Finish[n];
```

```
for (i=0; i<n; i++)
```

```
int count=0;
```

```
while (count < n) {
```

```
int found=0
```

```
for (i=0; i<n; i++)
```

```
{ if (Finish[i] == 0)
```

if (j == m)

for (k = 0; k < m; k++)

workA[k] += Alloc[i][k];

}

finish[i] = 1;

found = 1;

count++;

}

if (found == 0)

break;

}

int deadlock = 0

for (i = 0; i < n; i++)

if (finish[i] == 0)

deadlock = 1;

printf("Process P. d is deadlocked.\n", i);

}

if (deadlock == 0)

printf("In NoDeadlock, but dead\n");

return 0;

}

return 0;

}

Output:-

Enter no. of processes: 5

Enter no. of resources: 3

Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

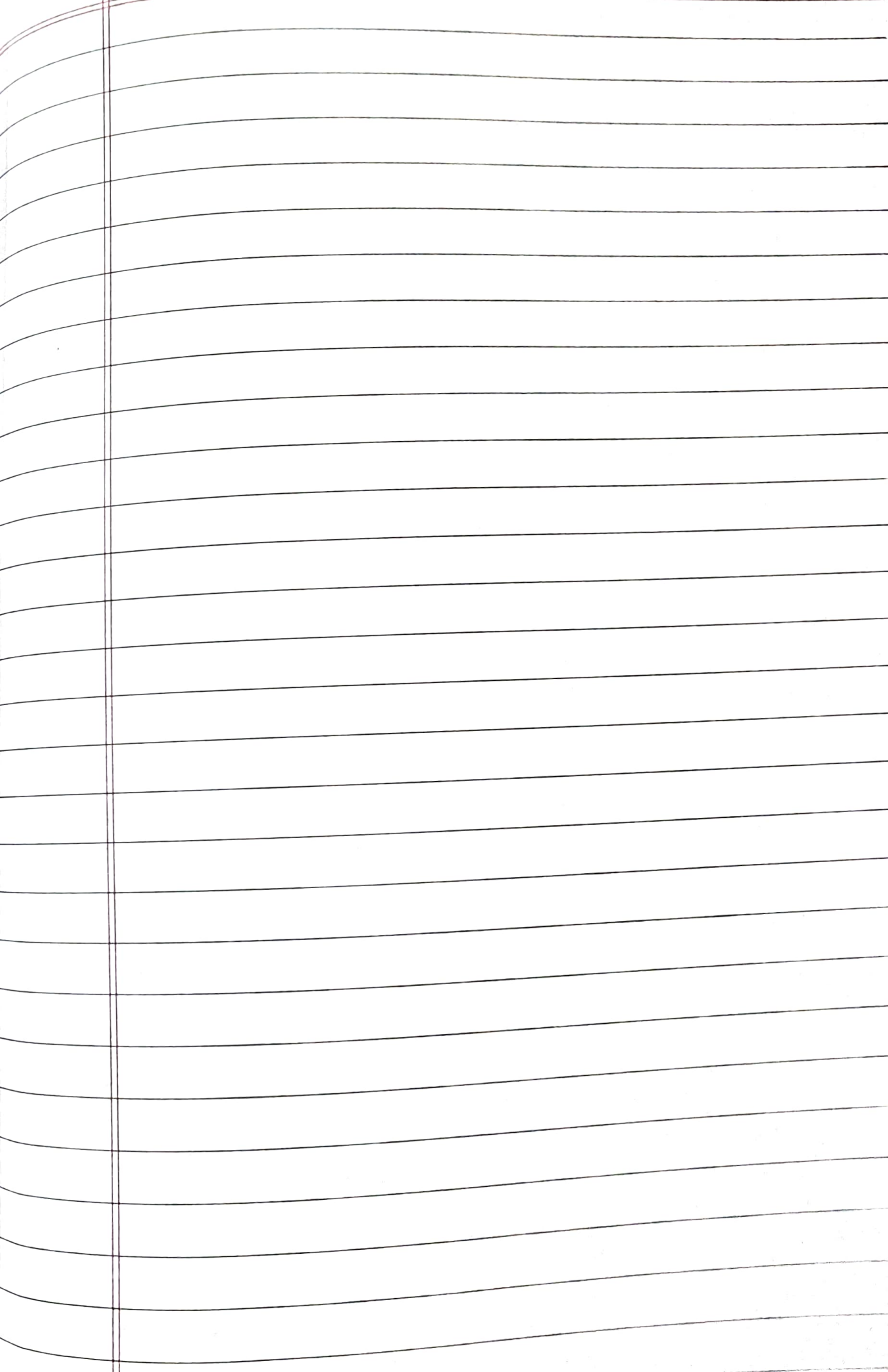
Available Resource:

0 0 0

Process P1 is ~~in deadlock~~ No deadlock detected.

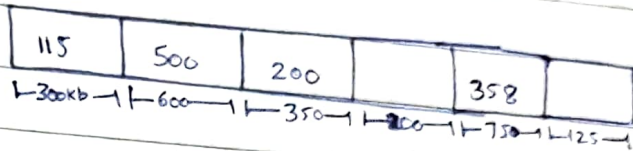
Process P3 is ~~in deadlock~~

Process P4 is ~~in deadlock~~



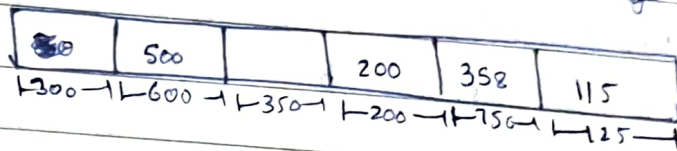
Q- Given 6 memory partitions of 300kb, 600kb, 350kb, 200kb, 750kb, 125kb (in order). Now would Best-fit, First-fit, Worst-fit Algorithms place processes of sizes 115kb, 500kb, 358kb, 200kb, 375kb (in order).

Ans- First-fit →



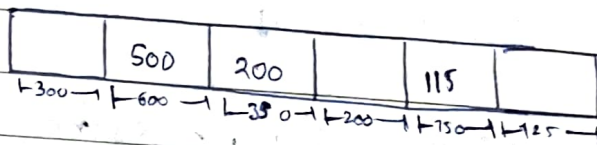
process with 375kb cannot be allocated with given memory

Best-fit →



process with 375kb cannot be allocated with memory

Worst-fit →



processes with 358 & 375 cannot be allocated with memory

write a C program to simulate the following
memory allocation techniques-

1) Worst-fit 2) Best-fit 3) First-fit

Code:-

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int np, i, j, nb;
```

```
    printf("Enter number of memory blocks:");
```

```
    scanf("%d", &nb);
```

```
    int blockSize[nb], block1[nb], block2[nb], block3[nb];
```

```
    printf("Enter sizes of %d memory blocks:\n", nb);
```

```
    for (i=0; i<nb; i++)
```

```
        scanf("%d", &blockSize[i]);
```

```
        block1[i] = blockSize[nb];
```

```
        block2[i] = blockSize[nb];
```

```
        block3[i] = blockSize[nb];
```

```
    }
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &np);
```

```
    int process[np];
```

```
    printf("Enter sizes of %d processes:\n", np);
```

```
    for (i=0; i<np; i++)
```

```
printf("First fit ---");  
printf("Process No. It Process Size It Block no.");  
for(i=0; i<np; i++) {  
    int allocated=0;
```

```
int firstfit[np], bestfit[np], worstfit[np];
```

```
for(i=0; i<np; i++) {  
    firstfit[i] = -1;  
    bestfit[i] = -1;  
    worstfit[i] = -1;  
}
```

```
int used1[nb];
```

```
for(i=0; i<nb; i++)
```

```
    used1[i] = 0
```

```
for(i=0; i<np; i++) {
```

```
    for(j=0; j<nb; j++) {
```

```
        if (!used1[j] && blockSize[j] >= process[i]) {
```

```
            firstfit[i] = j;
```

```
            used1[j] = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
int used2[nb];
```

```
for(i=0; i<nb; i++) {
```

```
    used2[i] = 0
```

```
for(i=0; i<np; i++) {
```

```
    for(j=0; j<nb; j++) {
```

```
        if (!used2[j] && blockSize[j] >= process[i]) {
```

```
if (best == -1 || blockSize[j] < blockSize[best]) {  
    best = j;
```

```
}  
if (best != -1) {  
    bestFit[i] = best;  
    used2[best] = 1;
```

```
}
```

```
at used3[nb];
```

```
for (i=0; i < nb; i++)  
    used3[i] = 0;
```

```
for (i=0; i < np; i++) {  
    int worst = -1;
```

```
    for (j=0; j < nb; j++) {
```

```
        if (!used3[j] && blockSize[j] >= process[i]) {
```

```
            if (worst == -1 || blockSize[j] < blockSize[worst]) {
```

```
                worst = j;
```

```
            }  
        }
```

```
    }
```

```
printf ("In --- First fit --- In");
```

```
printf ("Process No. In Process Size In Block No. In");
```

```
for (i=0; i < np; i++) {
```

```
    if (firstFit[i] != -1)
```

```
        printf ("%d\t%d\t%d\t", i+1, process[i], firstFit[i]  
                +1);
```

```
    else
```

```
        printf ("%d\t%d\t", i+1, process[i]);
```

```
    }
```

```

printf("\n --- Best fit --- \n");
printf("Process No. \t Process Size \t Block No. \n");
for (i=0; i<np; i++) {
    if (bestfit[i] != -1)
        printf("%d \t %d \t %d \n", i+1, procun[i], bestfit[i]+1);
    else
        printf("%d \t %d \t Not Allocated \n", i+1, procun[i]);
}

printf("\n --- Worst fit --- \n");
printf("Process No. \t Process Size \t Block No. \n");

for (i=0; i<np; i++) {
    if (worstfit[i] != -1)
        printf("%d \t %d \t %d \n", i+1, procun[i], worstfit[i]+1);
    else
        printf("%d \t %d \t Not Allocated \n", i+1, procun[i]);
}

return 0;
}

```

Output →

Enter number of memory blocks: 5

Enter size of 5 memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes:

212 417 112 426

--- First Fit ---

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

--- Best Fit ---

Process No.	Process Size	Block No.
2	212	4
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not Allocated.

18/10

Q- Write a C program to simulate page replacement algorithms.

a) FIFO b) LRU c) Optimal

→ #include <stdio.h>

```
int fifo(int p[], int n, int f){
    int fr[10], i, j, k=0, fault=0, found;

    for(i=0; i<f; i++) fr[i]=-1;

    for(i=0; i<n; i++){
        found=0;
        for(j=0; j<f; j++){
            if(fr[j]==p[i]) found=1;
        }

        if(!found){
            fr[k]=p[i];
            k=(k+1)%f;
            fault++;
        }
    }

    return fault;
}
```

```
int lru(int p[], int n, int f){
    int fr[10], time[10], i, j, ps, min, fault=0;

    for(i=0; i<f; i++){
        fr[i]=-1;
        time[i]=0;
    }

    for(i=0; i<n; i++){
        found=0;
        for(j=0; j<f; j++){
            if(fr[j]==p[i]) found=1;
        }

        if(!found){
            ps=-1;
            for(j=0; j<f; j++){
                if(fr[j]!=-1)
                    if(ps==-1 || time[j]<time[ps])
                        ps=j;
            }

            fr[ps]=p[i];
            time[ps]=i;
            fault++;
        }
    }

    return fault;
}
```

```

for (i=0; i<n; i++) {
    int found = 0;
    for (j=0; j<f; j++) {
        if (p[j] == p[i]) {
            found = 1;
            time[j] = i;
        }
    }
}

```

```

if (!found) {
    min = time[0];
    pos = 0;
    for (j=1; j<f; j++) {
        if (time[j] < min) {
            min = time[j];
            pos = j;
        }
    }
}

```

```

p[pos] = p[i];
time[pos] = i;
fault++;
}

```

```

return fault;
}

```

```

int optimal (int p[], int n, int f) {

```

```

    int fr[10], i, j, k, pos, farthest, fault = 0;

```

```

    for (i=0; i<f; i++)

```

```

        fr[i] = -1;

```

```
for (j=0; j<f; j++) {  
    if (fr[j] == p[i]) {  
        found = 1;  
        break;  
    }  
}
```

```
}
```

```
if (!found) {
```

```
    pos = -1;
```

```
    farthest = -1;
```

```
    for (k=i+1; k<n; k++) {
```

```
        if (fr[j] == p[k]) {
```

```
            nextUse = k;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (nextUse == -1) {
```

```
        pos = j;
```

```
        break;
```

```
    }
```

```
    if (nextUse > farthest) {
```

```
        farthest = nextUse;
```

```
        pos = j;
```

```
    }
```

```
}
```

```
fr[pos] = p[i];
```

```
fault++;
```

```
}
```

```
}
```

```

int main() {
    int p[20], n, f, i;

    printf("Enter no. of pages: ");
    scanf("%d", &n);

```

```

    printf("Enter pages: ");
    for (i=0; i<n; i++)
        scanf("%d", &p[i]);

```

```

    printf("Enter number of frames: ");
    scanf("%d", &f);

```

```

    printf("In FIFO Faults = %d", fifo(p, n, f));
    printf("In LRU Faults = %d", lru(p, n, f));
    printf("In Optimal Faults = %d", optimal(p, n, f));
}

```

Output →

Enter no. of pages: 15

Enter pages: 1 7 0 2 3 0 3 2 4 1 5 2 0 7 3

Enter no. of frames: 3

FIFO faults = 12

LRU Faults = 12

Optimal Faults = 10

~~88~~
15/2/6